

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS**

GIOVANNA REGINA RIBEIRO VITORINO

PROJETO FINAL QT CREATOR

Campos do Jordão
2026

GIOVANNA REGINA RIBEIRO VITORINO

PROJETO FINAL QT CREATOR

Relatório apresentado ao curso de Tecnologia em Análise e Desenvolvimento de Sistemas, referente à disciplina de Programação Orientada a Objetos, utilizando o framework Qt e a linguagem C++.

Orientador(a): Paulo Giovani de Faria Zeferino

Campos do Jordão
2026

RESUMO

Este relatório apresenta o processo de desenvolvimento de um jogo desenvolvido com a linguagem C++ utilizando o framework gráfico Qt. Trata-se de um jogo do gênero tiro espacial (shoot 'em up), no qual o jogador controla uma nave que se move horizontalmente e dispara projéteis contra inimigos gerados aleatoriamente, com o objetivo de eliminá-los antes que estes ultrapassem o limite inferior da tela. Ao longo do desenvolvimento, foram implementados sistemas essenciais para a construção da experiência interativa, incluindo a movimentação de elementos via temporizadores (QTimer), a reprodução de efeitos sonoros e trilha musical através dos módulos Qt Multimedia, um sistema de pontuação incrementado pela destruição de inimigos e um sistema de vidas decrementado quando inimigos escapam da área de jogo. O projeto aplica os princípios de Programação Orientada a Objetos (POO) estudados ao longo da disciplina, resultando em uma arquitetura modular na qual cada elemento do jogo é representado por uma classe independente, integrada ao ciclo de eventos da aplicação por meio do mecanismo de sinais e slots do Qt.

Palavras-chave: Jogo. Qt. Programação Orientada a Objetos. C++

RESUMO	2
1.1 Objetivos	3
1.2 Justificativa.....	3
1.3 Aspectos Metodológicos.....	3
1.4 Aporte Teórico.....	3
2 METODOLOGIA.....	3
Levantamento de Requisitos.....	3
Modelagem das Entidades do Jogo.....	3
Sinais, Slots e Temporização.....	3
Detecção de Colisão e Gerenciamento de Memória.....	3
2.1 Ferramentas Utilizadas.....	3
2.2 Modelagem UML.....	3
3 RESULTADOS OBTIDOS.....	3
3.1 Estrutura de Classes Implementadas	3
3.2 Trecho de Código Ilustrativo.....	3
3.3 Análise dos Resultados	3
4 CONCLUSÃO.....	3
4.1 Sugestões de Melhorias	3

REFERÊNCIAS3

1 INTRODUÇÃO

Este trabalho apresenta o desenvolvimento de um jogo desenvolvido em C++ utilizando o framework Qt, em sua versão baseada em Qt Widgets e no módulo Qt Graphics View. O projeto foi desenvolvido com o objetivo de reunir os conceitos de Programação Orientada a Objetos estudados em sala de aula, aplicando-os à construção de uma aplicação gráfica interativa em tempo real.

A mecânica do jogo é centrada na defesa de uma área contra uma invasão de inimigos: o jogador assume o controle de uma nave posicionada na parte inferior da tela, podendo deslocá-la horizontalmente e disparar projéteis verticais contra inimigos que surgem na parte superior da cena e avançam progressivamente. A cada inimigo destruído por um projétil, a pontuação do jogador é incrementada; caso um inimigo ultrapasse o limite inferior da tela sem ser destruído, uma unidade de vida é descontada do jogador. O sistema também conta com efeitos sonoros para o disparo e música de fundo, reproduzidos por meio dos recursos de multimídia do Qt.

Do ponto de vista técnico, o jogo é estruturado a partir do módulo Qt Graphics View (QGraphicsScene, QGraphicsView e QGraphicsItem), que fornece um modelo de cena para a gestão de elementos gráficos independentes, cada um deles encapsulado em uma classe própria. A movimentação dos elementos é orquestrada por temporizadores (QTimer) conectados, via sinais e slots, aos métodos responsáveis por atualizar a posição de cada item a cada intervalo de tempo.

1.1 Objetivos

O objetivo deste trabalho é demonstrar o processo de desenvolvimento de um jogo 2D, com interface gráfica, aplicando os conceitos fundamentais de Programação Orientada a Objetos por meio da linguagem C++ e do framework Qt. De forma específica, pretende-se: estruturar cada entidade do jogo (jogador, inimigo, projétil, pontuação e vidas) como uma classe independente, favorecendo encapsulamento e baixo acoplamento; utilizar o mecanismo de herança múltipla disponibilizado pelas classes do Qt (QObject e QGraphicsPixmapItem) para conciliar a capacidade de emitir e receber sinais com a capacidade de ser exibido graficamente em uma cena; empregar o mecanismo de sinais e slots do Qt para implementar comportamentos temporizados, como a movimentação automática de inimigos e projéteis; e integrar

recursos multimídia (efeitos sonoros e música) à experiência de jogo por meio dos módulos Qt Multimedia.

1.2 Justificativa

O desenvolvimento desse jogo justifica-se pela necessidade de aplicar, em um contexto prático, os conceitos teóricos de Programação Orientada a Objetos abordados ao longo da disciplina. A construção de jogos é amplamente reconhecida como uma estratégia pedagógica eficaz para o ensino de programação, pois exige a integração de diversos conceitos — estado, eventos, temporização, colisão e ciclo de vida de objetos — em um produto final tangível. Além do aspecto didático, o projeto permitiu o aprofundamento em particularidades do framework Qt, como o sistema de sinais e slots, a hierarquia de classes gráficas (QGraphicsItem) e os recursos de multimídia, conhecimentos diretamente aplicáveis ao desenvolvimento de aplicações desktop profissionais.

1.3 Aspectos Metodológicos

Para o desenvolvimento do jogo, foi utilizada a linguagem C++ no ambiente de desenvolvimento integrado Qt Creator, com o sistema de build qmake, conforme definido no arquivo de projeto (.pro). O framework Qt foi responsável pela criação da janela principal, pela renderização dos elementos gráficos por meio do módulo Graphics View, pela captura de eventos de teclado e pela reprodução de áudio através dos módulos Qt Multimedia (QMediaPlayer, QAudioOutput) e Qt Multimedia Widgets, sendo utilizado também o componente QSoundEffect para efeitos sonoros de curta duração. Os recursos gráficos e sonoros (imagens em formato PNG e áudios em formato MP3/WAV) foram incorporados ao executável por meio do sistema de recursos do Qt (arquivo .qrc), o que dispensa a distribuição de arquivos externos junto ao programa compilado. O desenvolvimento foi dividido em etapas sucessivas: inicialmente a estruturação da janela principal e da cena gráfica (classe Game); em seguida, a modelagem das entidades visuais do jogo (Player, Enemy, Bullet); na sequência, a implementação dos elementos de interface textual (Score e Health); e, por fim, a integração dos elementos sonoros e gráficos e o ajuste dos parâmetros de jogabilidade.

1.4 Aporte Teórico

O aporte teórico do projeto fundamenta-se nos quatro pilares clássicos da Programação Orientada a Objetos — encapsulamento, herança, polimorfismo e abstração — bem como na documentação oficial do framework Qt para a construção de interfaces gráficas e para a manipulação do módulo Graphics View. A organização das classes do projeto segue o padrão amplamente adotado em aplicações Qt, no qual elementos visuais dinâmicos herdam simultaneamente de `QObject` (para participar do mecanismo de sinais e slots) e de uma subclasse de `QGraphicsItem` apropriada ao tipo de conteúdo a ser exibido, como `QGraphicsPixmapItem` para imagens bitmap e `QGraphicsTextItem` para textos. Esse padrão de herança múltipla, peculiar ao Qt, exige a aplicação da macro `Q_OBJECT` e o uso de listas de inicialização que respeitem a ordem de declaração das classes-base, sendo abordado em detalhe na seção de modelagem do sistema.

2 METODOLOGIA

O desenvolvimento do projeto seguiu as seguintes etapas:

Levantamento de Requisitos

- Funcionais: movimentação horizontal do jogador (teclas direcionais); disparo de projéteis (tecla de espaço); geração periódica de inimigos em posição horizontal aleatória; detecção de colisão entre projétil e inimigo; sistema de pontuação incrementado a cada inimigo destruído; sistema de vidas decrementado quando um inimigo atravessa o limite inferior da tela.
- Não funcionais: renderização da cena por meio de QGraphicsView/QGraphicsScene; movimentação dos elementos via temporizadores QTimer com intervalo fixo de 50 milissegundos; reprodução de música de fundo em looping e de efeito sonoro de disparo sem bloqueio da interface gráfica.

Modelagem das Entidades do Jogo

Definição das entidades centrais do jogo através de Programação Orientada a Objetos:

- Player: representa a nave do jogador. Herda de QObject e QGraphicsPixmapItem, capturando eventos de teclado para movimentação e disparo, e reproduzindo o efeito sonoro do tiro por meio de um objeto QSoundEffect.
- Enemy: representa os inimigos. Após instanciado em posição horizontal aleatória, move-se verticalmente a intervalos regulares por meio de um slot conectado a um QTimer; ao atingir a parte inferior da tela, é removido da cena e provoca o decréscimo de uma vida.
- Bullet: representa os projéteis disparados pelo jogador. Move-se verticalmente para cima a cada intervalo de tempo e verifica, a cada movimento, colisões com os demais itens da cena por meio do método collidingItems(), identificando inimigos através de comparação de tipos em tempo de execução (typeid).

- Score e Health: classes derivadas de QGraphicsTextItem responsáveis, respectivamente, por exibir e atualizar a pontuação acumulada e o número de vidas restantes do jogador.
- Game: classe principal, derivada de QGraphicsView, responsável por inicializar a cena gráfica, instanciar o jogador, a pontuação e a barra de vidas, configurar o temporizador de geração de inimigos e iniciar a reprodução da música de fundo.

Sinais, Slots e Temporização

A movimentação autônoma dos elementos do jogo é implementada por meio do mecanismo de sinais e slots do Qt: cada classe dinâmica (Player, Enemy e Bullet) declara um slot move() (ou spawn(), no caso da geração de inimigos), conectado ao sinal timeout() de um QTimer próprio, instanciado no construtor de cada classe. Esse padrão dispensa o uso de um laço de jogo (game loop) manual, delegando à própria infraestrutura de eventos do Qt a responsabilidade de invocar periodicamente os métodos de atualização.

Deteção de Colisão e Gerenciamento de Memória

A detecção de colisões emprega o método collidingItems(), nativo de QGraphicsItem, que retorna a lista de itens graficamente sobrepostos ao item consultado dentro da mesma cena. No caso da classe Bullet, essa lista é percorrida a cada ciclo de movimentação em busca de um item cujo tipo, verificado em tempo de execução, corresponda à classe Enemy; identificada a colisão, ambos os itens são removidos da cena (removeItem()) e, em seguida, destruídos explicitamente (delete) para liberar a memória alocada dinamicamente, já que o Qt não realiza coleta automática de lixo.

2.1 Ferramentas Utilizadas

- Linguagem C++;
- Framework Qt (módulos Core, GUI, Widgets e Multimedia);
- Qt Creator (IDE e sistema de build qmake);
- Qt Graphics View Framework (QGraphicsScene, QGraphicsView, QGraphicsItem);

- Sistema de recursos Qt (arquivo .qrc) para empacotamento de imagens e sons;
- PlantUML, para a elaboração do diagrama de classes UML.

2.2 Modelagem UML

O diagrama de classes UML elaborado com o PlantUML, apresentado na Figura 1, evidencia as relações entre as principais entidades do sistema e as classes abstratas do próprio framework Qt. As classes Player, Enemy e Bullet realizam herança múltipla a partir de QObject e QGraphicsPixmapItem, conciliando o mecanismo de sinais e slots com a capacidade de renderização em uma QGraphicsScene; já as classes Score e Health derivam de QGraphicsTextItem, voltada à exibição de conteúdo textual na cena. A classe Game, por sua vez, herda de QGraphicsView e possui (relação de composição) um ponteiro para cada uma das demais entidades — scene, player, score e health —, centralizando a orquestração do jogo. As relações de criação dinâmica ("cria") indicam os pontos em que novos objetos são instanciados em tempo de execução: Player e MyRect criam objetos Bullet ao processar o evento de tecla de espaço, e ambos criam objetos Enemy no método spawn(), disparado periodicamente pelo temporizador definido na classe Game. As relações tracejadas identificadas como "acessa (extern)" representam o acoplamento por meio do ponteiro global game, declarado em main.cpp e utilizado pelas classes Bullet e Enemy para acessar os objetos score e health pertencentes à instância única de Game.

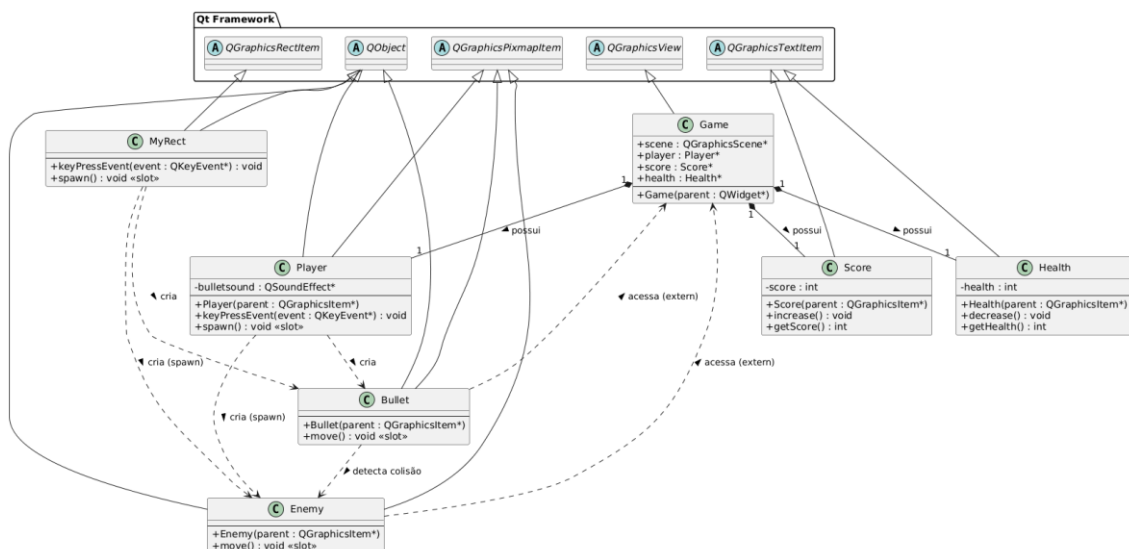


Figura 1 – Diagrama UML do projeto.

Observa-se ainda, no diagrama, a presença da classe `MyRect`, derivada de `QObject` e `QGraphicsRectItem`, que representa uma versão preliminar do jogador construída sobre um item gráfico retangular simples (sem textura), mantida no código-fonte como alternativa de implementação, mas não utilizada na instanciação efetiva do jogador dentro da classe `Game`, que emprega a classe `Player`, baseada em `QGraphicsPixmapItem` com sprite gráfico.

3 RESULTADOS OBTIDOS

Ao término do desenvolvimento, o StarDefender apresentou resultados funcionais em todos os módulos planejados. A seguir, são descritas as principais classes implementadas e o comportamento observado durante a execução do jogo.

Ao iniciar a aplicação, a classe Game cria a cena gráfica de 800 por 600 pixels, define a imagem de fundo, posiciona o jogador na parte inferior central da tela e inicializa os contadores de pontuação e de vidas, fixados respectivamente em zero e em três unidades. Em seguida, um temporizador é configurado para invocar o método spawn() do jogador a cada dois segundos, dando início à geração contínua de inimigos, e a trilha musical de fundo começa a ser reproduzida em looping através de um objeto QMediaPlayer.

Durante a partida, o jogador movimenta a nave horizontalmente por meio das teclas de seta, respeitando os limites laterais da cena (entre as posições 0 e 800 menos a largura do sprite), e dispara projéteis pressionando a barra de espaço, ação acompanhada da reprodução do efeito sonoro de tiro. Cada inimigo gerado surge em uma posição horizontal aleatória no topo da tela, rotacionado 180 graus em relação à sua imagem original, e desloca-se verticalmente para baixo a uma taxa constante de cinco pixels a cada cinquenta milissegundos.

Quando um projétil colide com um inimigo, a pontuação exibida na interface é incrementada em uma unidade e ambos os itens são removidos da cena e destruídos. Quando, ao contrário, um inimigo ultrapassa a coordenada vertical correspondente ao limite inferior da tela sem ser atingido, o contador de vidas é decrementado em uma unidade e o texto exibido na interface é atualizado de "Health: 3" para "Health: 2", e assim sucessivamente.

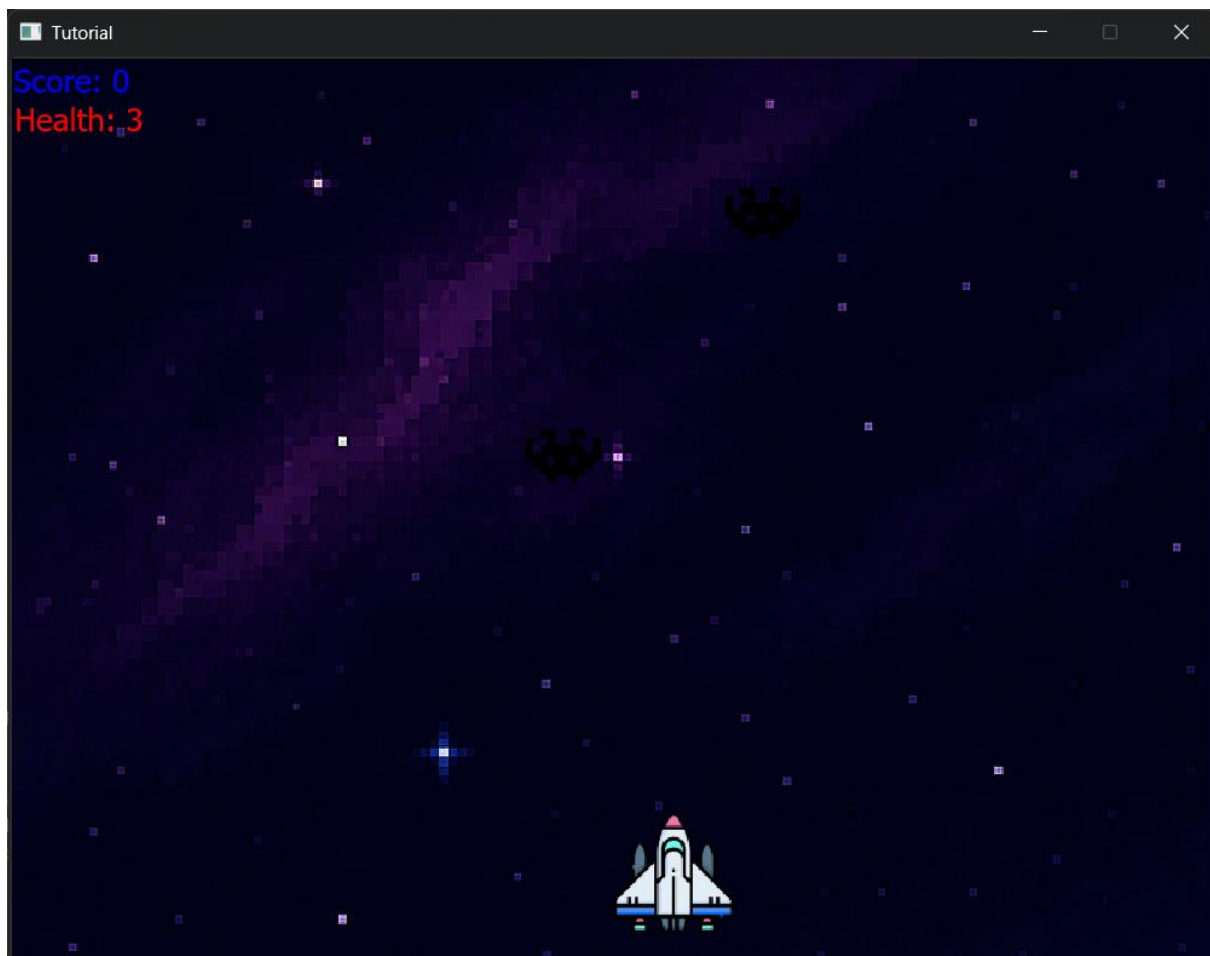


Figura 2 – Captura de tela.

3.1 Estrutura de Classes Implementadas

A Tabela 1 resume as principais classes do projeto, suas classes-base e suas responsabilidades dentro da arquitetura do sistema.

Classe	Classes-base (Qt)	Responsabilidade
Game	QGraphicsView	Orquestra a cena, o jogador, a pontuação, as vidas e a música de fundo.
Player	QObject, QGraphicsPixmapItem	Controla a nave do jogador: movimentação, disparo e geração de inimigos.
Enemy	QObject, QGraphicsPixmapItem	Representa o inimigo: posicionamento aleatório e movimentação vertical.
Bullet	QObject, QGraphicsPixmapItem	Representa o projétil: movimentação vertical e detecção de colisão.
Score	QGraphicsTextItem	Exibe e incrementa a pontuação do jogador.
Health	QGraphicsTextItem	Exibe e decrementa o número de vidas restantes.

Tabela 1 – Classes do sistema e responsabilidades.

3.2 Trecho de Código Ilustrativo

O trecho a seguir, extraído da classe Bullet, ilustra a lógica de detecção de colisão e a consequente atualização da pontuação e remoção dos itens envolvidos:

```
void Bullet::move(){
    QList<QGraphicsItem *> colliding_items = collidingItems();
    for (int i = 0, n = colliding_items.size(); i < n; ++i){
        if (typeid(*(colliding_items[i])) == typeid(Enemy)){
            game->score->increase();
            scene()->removeItem(colliding_items[i]);
            scene()->removeItem(this);
            delete colliding_items[i];
            delete this;
            return;
        }
    }
    setPos(x(), y()-10);
    if (pos().y() < 0){
        scene()->removeItem(this);
        delete this;
    }
}
```

Código 1 – Método move() da classe Bullet.

3.3 Análise dos Resultados

Os testes funcionais realizados ao longo do desenvolvimento confirmaram a correta operação dos subsistemas implementados. O sistema de movimentação do jogador respeitou consistentemente os limites laterais da cena, impedindo que a nave saísse da área visível. O mecanismo de disparo criou corretamente novos objetos Bullet a cada acionamento da barra de espaço, e a reprodução do efeito sonoro correspondente ocorreu sem perceptível atraso em relação ao evento de tecla.

O temporizador de geração de inimigos produziu novas instâncias de Enemy a intervalos regulares de dois segundos, distribuídas aleatoriamente ao longo do eixo horizontal da cena, e o deslocamento vertical desses inimigos ocorreu de forma constante até a saída da área de jogo ou até a colisão com um projétil. A verificação de colisão por meio de collidingItems() identificou corretamente as sobreposições entre projéteis e inimigos, incrementando a pontuação exibida na interface a cada acerto e evitando o vazamento de memória por meio da chamada explícita ao operador delete após a remoção de cada item da cena.

O sistema de vidas funcionou conforme o esperado, decrementando o contador exibido na interface sempre que um inimigo atravessava o limite inferior da tela sem

ser destruído. Observou-se, contudo, que o projeto, em seu estado atual, não implementa uma condição explícita de término de partida quando o número de vidas atinge zero, constituindo uma oportunidade de melhoria detalhada na seção de conclusão deste relatório.

4 CONCLUSÃO

O desenvolvimento do StarDefender permitiu a aplicação prática e integrada dos principais conceitos de Programação Orientada a Objetos no contexto da construção de uma aplicação gráfica interativa em C++ e Qt. Ao longo do projeto, foram exercitados o encapsulamento, por meio da definição de atributos privados acessados unicamente por métodos públicos das classes Score e Health; a herança, em especial a herança múltipla peculiar ao Qt, combinando QObject com diferentes subclasses de QGraphicsItem; o polimorfismo, evidenciado na verificação de tipos em tempo de execução durante a detecção de colisões; e a abstração, na representação de conceitos do domínio do jogo como classes independentes e coesas.

O uso do mecanismo de sinais e slots, articulado com instâncias de QTimer dedicadas a cada objeto dinâmico, mostrou-se uma alternativa eficaz à implementação manual de um laço de jogo, delegando à infraestrutura de eventos do Qt a responsabilidade pela atualização periódica do estado de cada entidade. De forma semelhante, o emprego do módulo Graphics View simplificou consideravelmente a renderização e a manipulação dos elementos visuais, ao oferecer nativamente operações como posicionamento, rotação, redimensionamento de pixmaps e detecção de colisão entre itens da cena.

Do ponto de vista da experiência de jogo, o sistema alcançou o objetivo de oferecer uma dinâmica simples e funcional de tiro e defesa, com feedback visual e sonoro imediato a cada ação do jogador, validando a viabilidade do framework Qt como ferramenta para o desenvolvimento de jogos bidimensionais educacionais.

4.1 Sugestões de Melhorias

Com base nas limitações identificadas durante o desenvolvimento e os testes, propõem-se as seguintes melhorias para versões futuras do projeto:

- implementação de uma condição explícita de término de partida (game over) quando o número de vidas atingir zero, interrompendo os temporizadores e exibindo uma tela de resultado final;
- inclusão de níveis de dificuldade progressiva, reduzindo o intervalo de geração de inimigos ou aumentando sua velocidade conforme a pontuação avança;

- persistência da pontuação mais alta (recorde) entre sessões de jogo, por meio de arquivo local ou do mecanismo QSettings do Qt;
- introdução de diferentes tipos de inimigos e de power-ups, ampliando a variedade de desafios apresentados ao jogador;
- refatoração do acesso às instâncias de Score e Health, atualmente realizado por meio de um ponteiro global externo, substituindo-o por injeção de dependência ou pelo mecanismo de sinais e slots, de modo a reduzir o acoplamento entre as classes Bullet, Enemy e Game.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. *NBR 14724*: informação e documentação – trabalhos acadêmicos – apresentação. Rio de Janeiro: ABNT, 2011.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. *NBR 6024*: informação e documentação – numeração progressiva das seções de um documento. Rio de Janeiro: ABNT, 2012.

QT GROUP. *Qt Documentation*. Disponível em: <https://doc.qt.io/>. Acesso em: 27 jun. 2026.

QT GROUP. *Qt for Desktop Development*. Disponível em: <https://www.qt.io/product/development-tools>. Acesso em: 27 jun. 2026.

QT GROUP. *Graphics View Framework*. Disponível em: <https://doc.qt.io/qt-6/graphicsview.html>. Acesso em: 27 jun. 2026.